

ZERO-VALUE SPLIT SPAM PERMANENTLY FILLS TreeOperator

A capital-free application-level DoS path against a fixed-capacity shared Merkle tree

TARGET	Zendex SC / hackenproof-public/zendex-sc
FINDING	Free circuit-valid zero-value split spam
CATEGORY	Application-Level Denial-of-Service (DoS)
FINAL ASSESSMENT	High — valid, previously unreported
DISCLOSURE	HackenProof DualDefense Audit — ZNDXSCDD-73
REPORT DATE	August 21, 2026

Economic Invariants First. Code Second.

Portfolio case study adapted from a submitted security report and its Foundry proof of concept. Prepared for professional portfolio presentation; not an official client audit report.

1. Executive Summary

TreeOperator exposes a hard lifetime leaf capacity derived from an immutable tree depth. The production deployment configuration uses depth 10, yielding 1,024 leaves. The split circuit enforces only the conservation equation $\text{amount}_b + \text{amount}_c = \text{amount}_a$ and does not require either output amount to be non-zero. This creates a legitimate, circuit-valid way to repeatedly carry the attacker's real value forward while minting a zero-value commitment on every split.

Each split consumes two tree slots. Starting from a minimal real deposit, an attacker can therefore consume the shared Merkle-tree capacity at negligible economic cost. Once the tree reaches MAX_LEAVES, TreeOperator.insert() reverts with TreeOperator__TreeFull(). Any protocol operation that depends on _insert() then fails closed for subsequent users.

The supplied Foundry PoC reproduces the mechanism against the real, unmodified contract path. It uses a depth-4 tree solely to reduce the demonstration to a handful of transactions; the mechanism scales linearly to the real depth-10 deployment. The only mocked component is proof verification inside the test harness. The finding itself does not depend on bypassing proof verification because the underlying Noir split circuit permits a zero-valued output.

HackenProof ultimately recorded the submission as a valid, previously unreported High finding rather than a duplicate. The report was closed because the program rewarded only Critical issues. The final assessment also narrowed the impact: affected insertion paths fail atomically and existing notes remain spendable, while an administrative release path remains available.

Attack capital	1 wei / minimal real deposit, plus gas
Demonstration	Depth-4 tree, 16 leaves; 7 split calls plus final filler
Production configuration	Depth 10; 1,024-leaf lifetime capacity
Core primitive	Split into (full value, zero value)
Capacity consumption	2 leaves per split
Failure condition	TreeOperator__TreeFull()
Final severity	High (HackenProof triage)

2. Finding

Free circuit-valid zero-value split spam permanently fills TreeOperator's fixed-capacity Merkle tree

Category: Application-Level Denial-of-Service (DoS)

The vulnerability is the combination of two design properties: (1) TreeOperator has an immutable maximum number of leaves for the lifetime of the deployed tree, and (2) split() can create a zero-value output commitment without violating the actual circuit constraints.

2.1 Fixed lifetime capacity

TreeOperator calculates MAX_LEAVES from the configured depth and rejects insertion once leafCount reaches that limit. The PoC confirms this is a hard ceiling with no demonstrated administrative function to expand or reset the tree.

```
MAX_LEAVES = 2 ** p.depth;
```

```
function insert(uint256 leaf) external onlyManager returns (uint256 epoch) {
    require(leafCount < MAX_LEAVES, TreeOperator__TreeFull());
    ...
}
```

```
}
```

2.2 Zero-value split primitive

The relevant split circuit constraint is:

```
assert(amount_b + amount_c == amount_a);
```

The supplied PoC documents that the circuit contains no corresponding non-zero constraint for `amount_b` or `amount_c`. Therefore a valid split can preserve the entire real amount in `commitment_b` while creating `commitment_c` with amount zero.

This is important because the attack does not require a malformed proof, a verifier bypass, a privileged role, a flash loan, or cooperation from another user. The proof can satisfy the real circuit constraints.

3. Attack Path

#	Stage	Mechanism
01	Seed	Attacker makes a minimal legitimate deposit and obtains a real commitment.
02	Split	Attacker splits the current commitment into (<code>amount_a</code> , 0).
03	Carry	The full real value continues in the non-zero child commitment.
04	Spam	The zero-valued child is still inserted into the shared Merkle tree.
05	Repeat	The carry-over commitment is split again, consuming two more leaves.
06	Exhaust	The attacker continues until the shared tree reaches <code>MAX_LEAVES</code> .
07	Brick	Future insertion attempts revert with <code>TreeOperator__TreeFull()</code> .

Invariant being abused: the protocol treats every inserted commitment as consuming scarce tree capacity, but the split circuit does not require every resulting commitment to carry economic value.

4. Technical Analysis

4.1 Shared resource and insertion point

The tree is shared across protocol operations that ultimately call the insertion path. The original submission identified deposit, swap, split, join, `addLiquidity`, `removeLiquidity`, and `createOrder/executeOrders` as relevant operations. Final triage refined this impact: split, join, swap, and `addLiquidity` hold consume and insert in one transaction; deposit inserts before `safeTransferFrom`; withdraw and `removeLiquidity` do not insert. Consequently, the affected operations fail atomically rather than silently losing user funds.

4.2 Demonstration configuration

```
uint8 constant TREE_DEPTH = 4; // demonstration only
...
depth: TREE_DEPTH
```

A depth-4 tree has $2^4 = 16$ leaves. The production deployment referenced in the submission uses `TREE_DEPTH = 10`, corresponding to 1,024 leaves. The shallow tree is therefore a controlled scale-down of the same capacity-exhaustion mechanism.

4.3 Proof-validation caveat

The PoC uses a `MockVerifier` because generating live Noir/Barretenberg proofs inside Foundry would require the external Noir/bb JavaScript toolchain. However, the PoC explicitly distinguishes this harness simplification from the vulnerability itself. The underlying circuit constraint permits `amount_c = 0`, so the attack does not rely on defeating the real verifier.

5. Proof of Concept

The supplied PoC is a compiled and passing Foundry test against the real, unmodified Zendex contract path. The test contract is TreeCapacityDosTest and the primary test is `test_freeZeroValueSplitSpamPermanentlyFillsTheTree()`.

5.1 Reproduction command

```
forge test --match-contract TreeCapacityDosTest -vv
```

5.2 Observed result

```
[PASS] test_freeZeroValueSplitSpamPermanentlyFillsTheTree()
Attacker's total real cost: 1 wei of USDT, plus gas.
Leaves used so far: 1
Splits performed: 7
Leaves used after spam: 15
Tree capacity: 16
... victim deposit -> TreeOperator__TreeFull()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

5.3 Core PoC logic

```
splitInputs[1] = bytes32(carryOverCommitment); // amount_b = full amount
splitInputs[2] = bytes32(spamCommitment);      // amount_c = 0

vaultManager.split(
    BaseManager.ProofData({proof: "", publicInputs: inclusionInputs}),
    BaseManager.ProofData({proof: "", publicInputs: splitInputs})
);
```

The PoC then fills the remaining slot if necessary and asserts that an unrelated victim's deposit reverts with the exact `TreeOperator__TreeFull` selector.

6. Impact Analysis

The principal impact is denial of future trustless operations that require insertion into the shared tree. The attack converts a scarce, irreversible protocol resource into a grieving target without requiring proportional economic value.

Availability	Shared Merkle-tree insertion capacity can be exhausted.
Attacker capital	Minimal real deposit; recurring cost is transaction gas.
User funds	Final triage found affected insertion paths fail atomically; existing notes remain spendable.
Recovery	Final triage noted an administrative release path, preserving a recovery route.
Scope	Protocol-wide effect on insertion-dependent operations, subject to path-specific behavior.
Reversibility	The demonstrated interface does not reset the consumed tree capacity.

Severity note: the original submission argued for broader permanent protocol-wide impact. HackenProof's final assessment was High rather than Critical because affected operations fail closed, existing notes remain spendable, and an administrative release path remains.

7. Recommended Remediation

7.1 Remove the free-leaf primitive

```
assert(amount_b != 0);
assert(amount_c != 0);
```

The immediate circuit-level mitigation proposed in the submission is to require both split outputs to be non-zero, mirroring the existing non-zero requirement on deposits. This prevents a split from producing a free zero-value

commitment solely to consume tree capacity.

7.2 Address the lifetime capacity assumption

The submission also recommends decoupling long-term protocol availability from a single immutable tree ceiling. Possible directions include a rolling or append-only forest of fixed-depth trees, or a planned migration mechanism before capacity exhaustion. Monitoring and alerting could provide warning before normal usage reaches the ceiling, but would not by itself remove the underlying hard limit.

8. Disclosure & Triage Outcome

Initial disposition	Closed as duplicate / out of scope
Corrected disposition	Accepted as a new, previously unreported finding
Final severity	High
Reward	No reward because program eligibility was limited to Critical findings
Reputation penalty	None

The triage correction is material: the final review distinguished this finding from F-2026-16602, which concerned $O(n)$ gas growth in `TreeOperator.insert()`. The final assessment stated that this finding was a distinct route to the fixed-capacity failure condition and superseded the earlier duplicate rationale.

9. Security Research Methodology Demonstrated

1	Map value flows	Identify where protocol operations consume and insert shared state.
2	Find hard ceilings	Check immutable capacity and recovery assumptions.
3	Test economic invariants	Ask whether every state transition carries the value the system assumes.
4	Challenge edge cases	Examine zero-value, dust, repeated transitions, and attacker-controlled sequencing.
5	Validate with Foundry	Use real entrypoints, real contract logic, and exact named errors.
6	Separate harness from bug	Document mocks and whether they are load-bearing for validity.
7	Assess practical impact	Trace affected paths, atomicity, user-fund behavior, and recovery.

Appendix A — Key Evidence from the Supplied PoC

Selected excerpts are included so the portfolio case study remains technically understandable without reproducing the entire attached test file.

```
// Demonstration uses depth 4 solely for speed.
uint8 constant TREE_DEPTH = 4;

// Carry the full value forward and create a zero-value leaf.
splitInputs[1] = bytes32(carryOverCommitment);
splitInputs[2] = bytes32(spamCommitment);

// Real split entrypoint.
vaultManager.split(...);

// Confirm the tree is completely full.
assertEq(tree.leafCount(), tree.MAX_LEAVES());

// Unrelated victim must fail with the real named error.
vm.expectRevert(ITreeOperator.TreeOperator__TreeFull.selector);
vaultManager.deposit(...);
```

The supplied PoC records a passing Forge run with one test executed and zero failures. It also states that the real ZendexVaultManager split path is exercised and that proof verification is the only mocked component.

Appendix B — Portfolio Note

This document is intentionally formatted as a client-facing security case study rather than a raw contest submission. It preserves the core technical mechanism, PoC evidence, final severity assessment, and remediation while making the work suitable as a Contra portfolio item.

Portfolio positioning: demonstrates smart-contract vulnerability research, DeFi threat analysis, economic-invariant reasoning, circuit-level review, Foundry PoC development, and severity/impact analysis.